

Assignment 10

Group Number – 6

- Swayam Borate (23110066)
- Aditya Borate (23110065)

Github Link : [CS203-STT-Labs/CS203_Lab_10 at main · adi776borate/CS203-STT-Labs](https://github.com/adi776borate/CS203-STT-Labs)

Task 1:

- Downloading the Data from Kaggle

```
▼ Downloading the Data

import kagglehub
import pandas as pd
import os

# Downloading the dataset
path = kagglehub.dataset_download('marius2303/ad-click-prediction-dataset')

print("Files in dataset:", os.listdir(path))
```

Files in dataset: ['ad_click_dataset.csv']

- Checking the Data

```
▼ Data Checking

[ ] df.head(5)
```

	id	full_name	age	gender	device_type	ad_position	browsing_history	time_of_day	click
0	670	User670	22.0	NaN	Desktop	Top	Shopping	Afternoon	1
1	3044	User3044	NaN	Male	Desktop	Top	NaN	NaN	1
2	5912	User5912	41.0	Non-Binary	NaN	Side	Education	Night	1
3	5418	User5418	34.0	Male	NaN	NaN	Entertainment	Evening	1
4	9452	User9452	39.0	Non-Binary	NaN	NaN	Social Media	Morning	0

- Checking the data-types of each column

▼ DataFrame Summary

This dataset contains 10,000 user interaction records with ads.

```
[ ] print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id               10000 non-null  int64
1   full_name        10000 non-null  object
2   age              5234 non-null   float64
3   gender           5307 non-null   object
4   device_type      8000 non-null   object
5   ad_position      8000 non-null   object
6   browsing_history 5218 non-null   object
7   time_of_day      8000 non-null   object
8   click            10000 non-null  int64
dtypes: float64(1), int64(2), object(6)
memory usage: 703.3+ KB
None
```

- Data Pre-processing
 - Dropping the two columns "id" and "full_name" as they are not useful for calculating the "proportions_ztest".

```
[ ] # The two columns "id" and "full_name" are not useful for calculating the proportions_ztest
df.drop(columns=['id', 'full_name'], inplace=True)
```

- The columns "age", "gender", "browsing_history" have around 5000 null values so they are being dropped.
- Also, the columns "time_of_day", "device_type", and "ad_position" have around 2,000 null values, so they are being replaced by their respective mode (i.e., the most frequently occurring value in each column) to handle missing data effectively.

```
# Drop columns with more than 4000 missing values
cols_to_drop = ['age', 'gender', 'browsing_history']
df.drop(columns=cols_to_drop, inplace=True)

# Fill mode for columns with 2000 missing values
for col in ['device_type', 'ad_position', 'time_of_day']:
    mode_value = df[col].mode()[0]
    df[col].fillna(mode_value, inplace=True)

# Confirm cleanup
print("Cleaned DataFrame:")
print(df.isnull().sum())
print(df.info())
```

- The data is cleaned now

```

Cleaned DataFrame:
id      0
full_name      0
device_type      0
ad_position      0
time_of_day      0
click      0
dtype: int64
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   id              10000 non-null   int64
1   full_name       10000 non-null   object
2   device_type     10000 non-null   object
3   ad_position     10000 non-null   object
4   time_of_day     10000 non-null   object
5   click           10000 non-null   int64
dtypes: int64(2), object(4)
memory usage: 468.9+ KB
None

```

- One-Hot Encoding the Categorical Variables
 - One-hot encoding on the categorical columns: "device_type", "ad_position", and "time_of_day". This process converts each category within these columns into separate binary (0/1) columns, making the data suitable.

```

import pandas as pd

categorical_cols = ['device_type', 'ad_position', 'time_of_day']
df = pd.get_dummies(df, columns=categorical_cols)
df.head()

```

- Splitting the data based on the Ad Position into Group_A and Group_B

```

# Group A: ad_position = 1 (Top)
group_a = df[df['ad_position_Top'] == 1]

# Group B: ad_position = 1 (Bottom)
group_b = df[df['ad_position_Bottom'] == 1]

```

- Getting the number of clicks(success) and total number of users in each group

```
[ ] from statsmodels.stats.proportion import proportions_ztest
import numpy as np

# Number of clicks (successes) in each group
count = np.array([group_a['click'].sum(), group_b['click'].sum()])
# Total number of users in each group
nobs = np.array([len(group_a), len(group_b)])

[ ] print("Group A - Clicks:", count[0], "Total Users:", nobs[0])
print("Group B - Clicks:", count[1], "Total Users:", nobs[1])
```

```
↗ Group A - Clicks: 1649 Total Users: 2597
Group B - Clicks: 3218 Total Users: 4817
```

- Calculating the P – value and Z – Score using from “statsmodels.stats.proportion import proportions_ztest”

```
[ ] from statsmodels.stats.proportion import proportions_ztest

z_stat, p_value = proportions_ztest(count, nobs)
print("Z-statistic:", z_stat)
print("P-value:", p_value)

if p_value < 0.05:
    print("There is a significant difference in click rates between the two groups.")
else:
    print("There is no significant difference in click rates between the two groups.")

↗ Z-statistic: -2.861975515593176
P-value: 0.004210094293323208
There is a significant difference in click rates between the two groups.
```

- Inference

Inference from Proportions Z-Test

The z-test compares the click-through rates between ads shown at the **top** (Group A) and the **bottom** (Group B) of the page.

- **Z-statistic:** -2.86
- **P-value:** 0.0042

Since the p-value is less than 0.05, we reject the null hypothesis.

Inference: There is a statistically significant difference in click rates between the two ad positions, indicating that ad position has an impact on user click behavior.

- How Z-score and P- value was calculated
 - **Null hypothesis means there is no difference** between the proportions of successes in the two groups — in other words, any observed difference is due to random chance.
 - **Alternative Hypothesis** suggests there **is** a significant difference between the two proportions.

2-Sample Z-Test for Proportions

To test whether there is a significant difference between the proportions of two groups, we use the following hypotheses:

- **Null Hypothesis (H_0):** $p_1 = p_2$
- **Alternative Hypothesis (H_1):** $p_1 \neq p_2$

Z-Test Statistic Formula:

$$z = \frac{\hat{p}_1 - \hat{p}_2}{\sqrt{\hat{p}(1 - \hat{p}) \left(\frac{1}{n_1} + \frac{1}{n_2} \right)}}$$

Where:

- $\hat{p}_1$ and $\hat{p}_2$ are the sample proportions for groups 1 and 2
- n_1 and n_2 are the sample sizes
- $\hat{p}$ is the **pooled proportion**, calculated as:

$$\hat{p} = \frac{x_1 + x_2}{n_1 + n_2}$$

Here, x_1 and x_2 are the number of successes (e.g., clicks) in each group.

P-Value:

- The p-value is then obtained from the **standard normal distribution** using the calculated (z)-value.
- A small p-value (typically < 0.05) indicates that the difference in proportions is statistically significant.

Task 2:

- Loading the Data

```
import pandas as pd
import numpy as np
from scipy.stats import ks_2samp

# Load the CSV files
train_df = pd.read_csv('train.csv')
test1_df = pd.read_csv('test1.csv')
test2_df = pd.read_csv('test2.csv')
```

- Combining the “train” , “test1” , “test2” for some pre-processing

```
# Combining dataframes to do some pre-processing

combined_df = pd.concat([train_df, test1_df, test2_df], axis=0, ignore_index=True)
```

- Converting object (string) data types to float by replacing commas (",") with dots ("."), then casting to float. As per the data description, values of **-200** are treated as missing and replaced with **Nan**. Additionally, non-informative columns such as 'Date', 'Time', 'Unnamed: 0', 'Unnamed: 15', and 'Unnamed: 16' are removed from the dataset.

```
# Converting object(string) to float
cols_to_fix = ['CO(GT)', 'C6H6(GT)', 'T', 'RH', 'AH']

for col in cols_to_fix:
    combined_df[col] = combined_df[col].astype(str).str.replace(',', '.', regex=False).astype(float)

# According to dataset description -200 suggests missing value
combined_df.replace(-200, np.nan, inplace=True)

# Removing non-useful and unnamed columns
combined_df.drop(columns=['Date', 'Time', 'Unnamed: 0', 'Unnamed: 15', 'Unnamed: 16'], inplace=True)
```

- Checking the summary of the combined data to check how many null values are there.

```
print("Summary of combined_df:")
combined_df.info()
```

Summary of combined_df:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4800 entries, 0 to 4799
Data columns (total 13 columns):

#	Column	Non-Null Count	Dtype
0	CO(GT)	3928 non-null	float64
1	PT08.S1(CO)	4715 non-null	float64
2	NMHC(GT)	914 non-null	float64
3	C6H6(GT)	4715 non-null	float64
4	PT08.S2(NMHC)	4715 non-null	float64
5	NOx(GT)	4118 non-null	float64
6	PT08.S3(NOx)	4715 non-null	float64
7	NO2(GT)	4115 non-null	float64
8	PT08.S4(NO2)	4715 non-null	float64
9	PT08.S5(O3)	4715 non-null	float64
10	T	4715 non-null	float64
11	RH	4715 non-null	float64
12	AH	4715 non-null	float64

dtypes: float64(13)
memory usage: 487.6 KB

- Dropping the NMHC(GT) column because it has too many missing values, For the remaining numeric columns, the missing values are filled with the median. Using the **median** helps avoid the influence of outliers.

```
# Dropping the NMHC(GT) column because it has too many missing values
combined_df.drop(columns='NMHC(GT)', inplace=True)

numeric_cols = combined_df.select_dtypes(include=['number']).columns
for col in numeric_cols:
    median_val = combined_df[col].median()
    combined_df[col].fillna(median_val, inplace=True)
```

- Again splitting them as per their index.

```
[ ] train_rows = len(train_df)
    test1_rows = len(test1_df)
    test2_rows = len(test2_df)

# Again split using slicing
train_df = combined_df.iloc[:train_rows].copy()
test1_df = combined_df.iloc[train_rows:train_rows + test1_rows].copy()
test2_df = combined_df.iloc[train_rows + test1_rows:].copy()
```

- Now each of the data frame train, test1, test2 is cleaned and has no null values.
- Getting all the numeric columns from the dataset and creating dictionaries to store the KS test results then KS test is then applied separately between each column of the training set and test1, and between the training set and test2, using “scipy.stats.ks_2samp”.

```
[ ] # Get all numeric columns common to all datasets
numeric_cols = list(set(train_df.select_dtypes(include='number').columns)
                    & set(test1_df.select_dtypes(include='number').columns)
                    & set(test2_df.select_dtypes(include='number').columns))

# KS test results storage
ks_results_test1 = {}
ks_results_test2 = {}

for col in numeric_cols:
    train_col = train_df[col]
    test1_col = test1_df[col]
    test2_col = test2_df[col]

    # KS test
    ks1 = ks_2samp(train_col, test1_col)
    ks2 = ks_2samp(train_col, test2_col)

    ks_results_test1[col] = (ks1.statistic, ks1.pvalue)
    ks_results_test2[col] = (ks2.statistic, ks2.pvalue)
```

- Results
 - Code to display the results.

```
[ ]
print("\nKS Test Results: Test1 vs Train")
print("-" * 50)
for col, (stat, pval) in ks_results_test1.items():
    print(f"{col:<20} | KS Statistic = {stat:8.4f} | p-value = {pval:8.4f}")

print("\nKS Test Results: Test2 vs Train")
print("-" * 50)
for col, (stat, pval) in ks_results_test2.items():
    print(f"{col:<20} | KS Statistic = {stat:8.4f} | p-value = {pval:8.4f}")
```


KS Test Results: Test1 vs Train

RH	KS Statistic =	0.0191	p-value =	0.9722
PT08.S1(CO)	KS Statistic =	0.0328	p-value =	0.4900
C6H6(GT)	KS Statistic =	0.0228	p-value =	0.8885
PT08.S2(NMHC)	KS Statistic =	0.0234	p-value =	0.8686
NO2(GT)	KS Statistic =	0.0166	p-value =	0.9940
PT08.S5(O3)	KS Statistic =	0.0300	p-value =	0.6057
AH	KS Statistic =	0.0259	p-value =	0.7764
PT08.S4(NO2)	KS Statistic =	0.0219	p-value =	0.9154
NOx(GT)	KS Statistic =	0.0175	p-value =	0.9885
T	KS Statistic =	0.0184	p-value =	0.9799
CO(GT)	KS Statistic =	0.0259	p-value =	0.7764
PT08.S3(NOx)	KS Statistic =	0.0344	p-value =	0.4304

KS Test Results: Test2 vs Train

RH	KS Statistic =	0.1797	p-value =	0.0000
PT08.S1(CO)	KS Statistic =	0.1275	p-value =	0.0000
C6H6(GT)	KS Statistic =	0.1619	p-value =	0.0000
PT08.S2(NMHC)	KS Statistic =	0.1613	p-value =	0.0000
NO2(GT)	KS Statistic =	0.4075	p-value =	0.0000
PT08.S5(O3)	KS Statistic =	0.1366	p-value =	0.0000
AH	KS Statistic =	0.4213	p-value =	0.0000
PT08.S4(NO2)	KS Statistic =	0.6166	p-value =	0.0000
NOx(GT)	KS Statistic =	0.5241	p-value =	0.0000
T	KS Statistic =	0.4919	p-value =	0.0000
CO(GT)	KS Statistic =	0.1197	p-value =	0.0000
PT08.S3(NOx)	KS Statistic =	0.3422	p-value =	0.0000

• Inference

Test1 vs Train

Most features have:

- **KS statistic < 0.04**
- **p-value > 0.4** (many close to 1)

Interpretation: The distributions of features in **test1** are **very similar** to those in the training set. Hence, **no significant covariate shift** is observed.

Test2 vs Train

All features have:

- **KS statistic > 0.1**, some as high as **0.6**
- **p-value = 0.0000** (highly significant)

Interpretation: The distributions of **test2** differ significantly from the training set across all features. This strongly suggests a **covariate shift** between **test2** and the training data.

Final Conclusion

- **Test1** does **not** exhibit covariate shift (high p-values, low KS stats).
- **Test2** does exhibit covariate shift (low p-values, high KS stats).

Therefore, `test2.csv` exhibits covariate shift relative to `train.csv`.

- **Kolmogorov–Smirnov test**

- ✓ **Kolmogorov-Smirnov (KS) Test: Calculating Statistic and p-value**

The KS test compares the distributions of a feature in the training and test sets to detect covariate shift.

KS Statistic (D):

$$D = \sup_x |F_1(x) - F_2(x)|$$

- $F_1(x)$: Empirical cumulative distribution function (CDF) of the feature in the training set
- $F_2(x)$: Empirical CDF of the feature in the test set
- $\sup_x$: Maximum difference over all values of (x)

p-value:

- The p-value is computed based on the KS statistic (D) and the sizes of the two samples.
- It represents the probability of observing a KS statistic as extreme as (D) under the null hypothesis (that both samples are drawn from the same distribution).
- **KS Statistic (D):** Measures the largest difference between the two CDFs.
- **p-value:** Indicates if the distributions are significantly different (small p-value means significant difference).